ChefIA

Rails + Inteligência Artificial na prática

Workshop Rails Girls 🤝 Le Wagon

Eu me chamo **Giovanna Montemurro** e atualmente trabalho como engenheira de software no Nubank.

Depois de 15 anos como jornalista, eu mudei de profissão após o bootcamp do Le Wagon.

🍳 O que vamos construir?

ChefIA — uma aplicação web onde você:

1. Digita os ingredientes que tem na geladeira
2. Clica em **"Gerar receita com IA"**
3. Recebe duas sugestões de receitas personalizadas

> ✨ Vamos ver o app funcionando agora — e depois entender como ele foi feito!

🌐 O que é uma aplicação web?

Pensa assim: quando você acessa o Instagram, o iFood ou o Google...

- Você digita um endereço no navegador
- O navegador manda um **pedido** para um computador em algum lugar do mundo
- Esse computador processa o pedido e manda de volta uma **resposta** (a página que você vê)

> Esse computador que fica "escutando" e respondendo pedidos é chamado de **servidor**. O código que roda nele é o **backend** da aplicação.

💎 O que é Ruby on Rails?

Ruby é uma linguagem de programação criada no Japão nos anos 90.
É conhecida por ser elegante e fácil de ler — quase como inglês.

Rails é um *framework* construído em Ruby — pense como uma caixa de ferramentas gigante que já vem organizada, com tudo que você precisa para construir um app web sem reinventar a roda.

Com Rails, você consegue:

- Criar páginas web
- Salvar informação no banco de dados
- Conectar tudo em poucas linhas

Foram construídos em Rails:

- Airbnb
- Twitter
- Github

🏗️ Como o Rails organiza um app?

Rails segue um padrão chamado **MVC** — Model, View, Controller.

Parte	O que faz
Model	Cuida dos dados
View	O que aparece na tela
Controller	A lógica, o que acontece ao clicar

> Quando você clica em "Gerar receita", o **Controller** recebe o pedido, chama a IA, salva o resultado no **Model** e manda a resposta de volta pra **View**.

🔄 O que é CRUD?

A maioria dos apps web faz as mesmas quatro operações básicas:

Operação	O que faz	Exemplo no ChefIA
Create	Criar	Gerar uma nova receita
Read	Ler	Ver uma receita gerada
Update	Atualizar	Editar os ingredientes
Delete	Deletar	Remover uma receita

> Hoje vamos construir o **C** e o **R** — criar e visualizar receitas. O projeto completo no GitHub tem o CRUD inteiro para você explorar em casa!

📦 O que são Gems?

No mundo Ruby, bibliotecas de código prontas se chamam **Gems**.

É como uma loja de apps: alguém já resolveu um problema, empacotou a solução e disponibilizou para todo mundo usar.

```
# Gemfile — a lista de "apps" do seu projeto
gem "rails"          # o framework
gem "ruby_llm"       # conexão com IAs
gem "pg"             # banco de dados PostgreSQL
```

Rodando `bundle install`, o Rails baixa e instala todas as gems listadas no `Gemfile`.

🤖 O que é IA / LLM?

LLM = *Large Language Model* (Grande Modelo de Linguagem)

É o tipo de IA por trás do **ChatGPT**, **Google Gemini** e **Claude**.

Esses modelos foram treinados em quantidades enormes de texto e aprenderam a:

- Entender linguagem humana
- Gerar respostas coerentes e criativas
- Traduzir, resumir, programar, cozinhar 👩‍🍳...

> A gente não precisa *criar* uma IA — a gente **conversa** com ela via API e usa as respostas dentro do nosso app.

🔌 O que é uma API?

Imagina que a IA é um restaurante estrelado.

Você não entra na cozinha — você **faz um pedido** pelo garçom.

A **API** é esse garçom: você manda uma mensagem (o prompt), ela leva até a IA, e traz de volta a resposta.

> Para usar a API, você precisa de uma **chave de API** — é como um crachá que identifica quem está fazendo o pedido.

🪙 O que são Tokens?

A IA não lê palavras — ela lê **tokens**.

Um token é, em média, **~4 caracteres** de texto.

> "Ruby on Rails" → ~3 tokens
> Um parágrafo de 200 palavras → ~270 tokens

- Cada chamada à API consome tokens
- Tokens são o que determina o **custo** da chamada
- Cada modelo tem um limite máximo chamado **context window**

> Para o nosso app, cada receita gerada consome pouquíssimos tokens — é bem barato!

🧩 Antes do código: alguns conceitos

Termo	O que é	Exemplo
Variável	Um nome que guarda um valor	`receita = "omelete"`
String	Um texto entre aspas	`"ovo, queijo, tomate"`
Método	Um bloco de código com um nome	`def criar_receita`
Classe	Um molde que agrupa dados e métodos	`RecipeSuggestion`

> Não precisa memorizar — é só para você reconhecer quando aparecer na tela! 👀

Hora da Demo! Vamos construir a ChefIA passo a passo

⚙️ Setup

Criando o projeto com o template Le Wagon:

```
rails new \
  -d postgresql \
  -m https://raw.githubusercontent.com/lewagon/rails-templates/master/minimal.rb \
  chefIA
cd rails-girls-chefIA
```

Adicionando a gem RubyLLM ao `Gemfile`:

```
gem "ruby_llm", "~> 1.2.0"
```

```
bundle install
```

⚙️ Configurando a gem de IA

Criando o arquivo de configuração:

```
touch config/initializers/ruby_llm.rb
```

```
# config/initializers/ruby_llm.rb
RubyLLM.configure do |config|
  config.openai_api_key = ENV["OPENAI_API_KEY"]

  # Para usar GitHub Models (gratuito):
  # config.openai_api_key = ENV["GITHUB_TOKEN"]
  # config.openai_api_base = "https://models.inference.ai.azure.com"
end
```

⚙️ Banco de dados

Gerando o Model e a Migration:

```
rails generate model RecipeSuggestion ingredients:text recipe:text
rails db:migrate
```

Isso cria uma tabela no banco com duas colunas:

- `ingredients` — o que o usuário digitou
- `recipe` — a receita gerada pela IA

⚙️ Rotas

```ruby
# config/routes.rb
Rails.application.routes.draw do
  root to: "recipe_suggestions#new"
  resources :recipe_suggestions, only: [:new, :create, :show]
end
```

```
rails routes | grep recipe_suggestions
```

Rota	O que faz
`GET /recipe_suggestions/new`	Exibe o formulário
`POST /recipe_suggestions`	Processa e gera a receita
`GET /recipe_suggestions/:id`	Exibe a receita gerada

⚙️ Controller - ação new

```ruby
class RecipeSuggestionsController < ApplicationController
  def new
    @suggestion = RecipeSuggestion.new
  end
end
```

⚙️ View - new

Formulário (`new.html.erb`):

```
<h1>👩 ChefIA, o que eu posso cozinhar?</h1>

<%= simple_form_for @suggestion do |f| %>
  <%= f.input :ingredients,
        label: "Quais ingredientes você tem?",
        input_html: { rows: 5 } %>

  <%= f.button :submit, "Sugerir receita" %>
<% end %>
```

⚙️ Controller - ação create

```ruby
class RecipeSuggestionsController < ApplicationController
  def create
    @suggestion = RecipeSuggestion.new(suggestion_params)
    response = RubyLLM.chat.ask("Sugira receitas com esses ingredientes: #{@suggestion.ingredients}")
    @suggestion.recipe = response.content

    if @suggestion.save
      redirect_to @suggestion
    else
      render :new, status: :unprocessable_entity
    end
  end

  private

  def suggestion_params
    params.require(:recipe_suggestion).permit(:ingredients)
  end
end
```

📝 O prompt faz toda a diferença

Veja a diferença entre um prompt simples e um estruturado:

❌ Prompt simples

```
"Sugira receitas com: #{ingredientes}"
```

Resultado vago, sem estrutura, pode ignorar o que você pediu.

✅ Prompt estruturado

```
Você é um chef experiente.
Sugira DUAS receitas com:
#{ingredientes}

Receita 1: só com esses ingredientes.
Receita 2: com poucos extras.

Para cada uma: nome, ingredientes,
passo a passo. Use emojis. 🍳
```

⚙️ O Prompt completo

```
def build_prompt(ingredients)
  <<~PROMPT
    Você é um chef de cozinha experiente e criativo.
    Sugira duas receitas simples com base nos seguintes ingredientes:

    #{ingredients}

    Receita 1: Deve usar APENAS os ingredientes listados acima.
    Receita 2: Pode incluir alguns ingredientes adicionais.

    Para cada receita, retorne:
    - Nome da receita
    - Lista de ingredientes
    - Instruções passo a passo

    Mantenha simples e amigável para iniciantes.
    Formate com emojis e seções claras.
  PROMPT
end
```

⚙️ Controller - ação show

```ruby
class RecipeSuggestionsController < ApplicationController
  def show
    @suggestion = RecipeSuggestion.find(params[:id])
  end
end
```

⚙️ View - show

Resultado (`show.html.erb`):

```
<h2>Seus ingredientes</h2>

<p><%= simple_format @suggestion.ingredients %></p>

<h2>Receita sugerida</h2>

<pre><%= @suggestion.recipe %></pre>
```

🔐 Segurança — o que você precisa saber

Sua chave de API é como a senha do seu cartão de crédito.

Se ela vazar no GitHub, alguém pode usá-la e **você paga a conta**.

```
# Crie um arquivo .env na raiz do projeto
OPENAI_API_KEY=sua_chave_aqui
```

⚠️ O arquivo `.env` **NUNCA** deve ir para o GitHub. Verifique se ele está no `.gitignore`!

No projeto ChefIA, o `.gitignore` já está configurado. Você está protegida. ✅

🔐 Desative o auto-recharge

Na plataforma da OpenAI, existe uma opção chamada **Auto recharge** — ela recarrega seu crédito automaticamente quando acaba.

Recomendamos desativar para ter controle total do gasto.

Onde encontrar: **Settings** → **Billing** → **Auto recharge**

Você também pode definir um **limite de gasto mensal** para nunca ter surpresas.

🆓 Como conseguir uma chave gratuita

A OpenAI exige cartão de crédito. Mas existe uma alternativa **100% gratuita**: o **GitHub Models**.

Com o seu token do GitHub, você acessa os mesmos modelos de IA sem pagar nada.

> ⚠️ Tem limites de uso (rate limits) e funciona só em inglês — perfeito para estudar!

Sugestões para seguir com o projeto

- Criar um índice de sugestões de receitas já criadas
- Permitir que o usuário delete sugestões de receitas
- Permitir que o usuário edite os ingredientes em uma sugestão
- Melhorar o frontend, deixar tudo mais bonito

📚 Onde continuar aprendendo

Ruby on Rails:

- guides.rubyonrails.org — documentação oficial
- theodinproject.com — curso gratuito completo

Le Wagon:

- lewagon.com/pt-BR/events — workshops gratuitos

Comunidade:

- stackoverflow.com — dúvidas técnicas
- Sua IA favorita — use para **aprender**, não só para copiar código! 🤖

👩‍🍳 É isso!

O código completo da ChefIA está disponível em:

github.com/giovannasm/chefIA

Dúvidas? 💬

Obrigada por estar aqui! 🙏

Giovanna Montemurro

giovannasm@gmail.com

https://www.linkedin.com/in/giovannasm/